Take-Home Assignment

Lead AI ML + Full Stack Engineer (with GenAI)

Document Verification System

Time budget	3 days
Stack	Your choice (see Technical Requirements)
Submission	Public/private GitHub repo
Domain	Medical — prescribed medications verified against an institutional formulary
Weighting	50% AI / GenAI, 50% Full-Stack

1. Task Overview

Build a document verification system that ingests a primary document and a reference document, generates a summary of the primary document, surfaces critical points from the primary document that the patient should take notice of, extracts domain entities (prescribed medications with their dose, frequency, duration, and indication) from it, and verifies each entity against the reference document using retrieval-augmented generation. The user should see the primary document alongside the generated summary, critical-point list, and a flagged-issues panel where each flag is grounded in a citation to the reference.

This assignment is deliberately under-scoped for the time budget. We do not expect you to finish everything. We are evaluating your judgment about what to build, how to architect it, and what to defer alongside the quality of what you do ship.

2. What You Are Building

2.1 Core flow

1. User signs in (basic auth — see §3.4).
2. User uploads a primary document (PDF or DOCX) and selects a reference document. For this assignment we provide one reference document; the system should be designed so additional references could be added later.
3. Backend runs an asynchronous pipeline. The frontend shows live status as each stage completes: parsed → chunked → embedded → summarized → key points extracted→ entities extracted → verified → done.

4. When processing completes, the user sees a document view: the primary document rendered on one side; on the other side, a summary of the document, a list of critical points the patient should take notice of, and a flagged-issues panel.
5. Each flagged issue must cite the specific passage in the reference document that contradicts the primary document (or note explicitly that no supporting passage exists). Citations should surface the underlying reference text. Citation should be done to the maximum reference level possible (Example: page number, section number).

2.2 What "verification" means

The reference document is the ground truth for how drugs should be prescribed at the institution. The system must figure out the relationship between the two documents: extract entities from the primary, look them up in the reference, and check whether the prescribed parameters match the formulary's prescribing parameters. In other words, the primary document makes claims (prescriptions with specific doses, frequencies, and durations); the reference document is the authoritative source of truth (formulary monographs with the correct prescribing parameters); and the system verifies those claims against the source of truth.
For each medication extracted from the primary document, the system must determine one of three states:

- **Supported** — the reference contains a monograph for the prescribed medication and the prescribing parameters (dose, frequency, duration, route, timing) are consistent with what the reference states.
- **Contradicted** — the reference contains a monograph for the prescribed medication but one or more prescribing parameters conflict with what the reference states (wrong dose, wrong frequency, wrong duration, wrong timing of administration, contraindicated indication, etc.).
- **Unsupported** — the reference contains no monograph for the prescribed medication.

You decide how to model and surface this. Be opinionated and explain your reasoning in the README.

3. Technical Requirements

These are must-haves. Anything beyond this is stretch (see §4). If something here is at risk, cut a stretch goal first — not a must-have.

3.1 GenAI

- **A single summary** of the primary document generated by an LLM. The summary should give the reader a faithful overview of the document's content. You decide the prompt structure, target length, and tone.

- **A list of critical points** the patient should take notice of, extracted from the primary document. *These are items the patient must not miss — for example, a medication that must be taken at a specific time, a duration limit, a side effect they must report immediately, or a follow-up appointment. The critical points must be drawn from content that exists in the primary document; the system must not invent new clinical recommendations or add advice that is not present in the document. You decide how to define and rank "critical" and justify the choice in the README.*
- Structured entity extraction: pull prescribed medications from the primary document as structured records (drug name, dose, unit, route, frequency, duration, indication, source page). Reliable JSON output is the bar.
- **Verification**: for each extracted entity, the system retrieves relevant reference passages and prompts the LLM to classify as Supported / Contradicted / Unsupported with an explanation grounded in the retrieved passage. Be opinionated about how to compare prescribing parameters — checking only that the drug name exists in the reference is not sufficient; the system should compare dose, frequency, duration, and timing against the reference monograph.
- Hallucination control: the verification output must cite the reference passage it relied on. If no passage was retrieved with sufficient relevance, the system should say so rather than guess. The same standard applies to the critical-points list.

3.2 RAG

- Chunk the reference document, generate embeddings, and store them in a vector store. Justify your chunking strategy and embedding model choice in the README. Note that the reference is organized as discrete monographs, your chunking strategy should take that structure into account.
- Targeted retrieval for verification: for each extracted entity, retrieve the most relevant reference passages.
- Be explicit about your retrieval evaluation strategy in the README.

3.3 Backend

- **Asynchronous processing is non-negotiable.** The upload endpoint must return immediately with a job ID. Blocking the HTTP request on the pipeline is an automatic deduction.
- Pipeline status must be observable in real time from the frontend. Server-sent events or WebSockets are both acceptable. Polling is a fallback, not a primary choice — if you choose polling, justify it.

- The pipeline must have granular, named stages (see §2.1). Each stage's status and timing should be queryable and visible to the frontend.
- Per-stage error handling: a failure at any stage must be captured, surfaced to the frontend, and not leave the job in a hung state.
- Documents belong to a user. A user cannot read another user's documents through any endpoint.
- When a user uploads a document, all records created as part of that upload must be written atomically. A partial write (where some records are created and others are not) must not be possible. If any part of the creation fails, the system must roll back to a clean state. Document in your README which operations you grouped into a transaction and why.

3.4 Database

- PostgreSQL with pgvector. Design the schema yourself; document it in the README (an ERD or annotated SQL is fine).
- Migrations should be versioned.

3.5 Frontend

- Use ReactJS with optimistic UI design pattern.
- Login / signup.
- Upload page with drag-and-drop and file validation.
- Real-time pipeline visualization while the document processes. The user should see each stage advance, with timestamps, and should be able to refresh the page mid-processing without losing visibility into where the job is.
- Document view: primary document rendered (PDF.js, react-pdf, or equivalent) on one side; the document summary, critical-points list, and flagged issues on the other.
- **Clickable citations.** A flagged issue must let the user see the supporting reference passage. A clear preview panel is sufficient, jumping into the reference document itself is an extra credit point.
- Loading, empty, and error states are required, not optional.

3.6 Auth

Email + password is fine. JWT or session cookies are both fine. We are not evaluating password reset flows, 2FA, OAuth, or anything beyond the basics; keep it simple and correct.

3.7 Logs and Test Cases

- **Logs**: the system should produce structured, meaningful logs for each pipeline stage (ingestion, extraction, retrieval, verification). Logs should be sufficient for a reviewer to

trace a single document's processing end to end and diagnose where a failure or unexpected result occurred.

- **Test cases**: include targeted tests around the verification logic (e.g., does a known-contradicted medication get flagged correctly?) and any authorization checks. Broad coverage is not expected within the time budget, but the tests you do write should demonstrate how you think about correctness.

3.8 Packaging

- Single docker-compose.yml that brings up the full stack: database, backend, frontend, any worker processes.
- README with setup instructions a stranger could follow in under ten minutes.
- Seed data or seed script so the reviewer can log in immediately without going through signup.

4. Extra Credit

List in your README what you attempted and what you would have done differently with more time.

- Deep-link citations into the reference document. When a flagged issue cites a reference passage, provide a "View in reference document" option that navigates the user to the quoted text within the rendered reference document.
- Duplicate upload detection. If the same user uploads the same document twice within a configurable time window (default: 5 minutes), the system should return the existing job rather than creating a new one. Document your approach to identifying duplicates (e.g., content hash) and how the time window is configured.

5. Provided Assets

- **primary_document.docx** - Hospital discharge summary listing prescribed medications for an imaginary patient. This is the document your system will analyze. It contains deliberate prescribing errors.
- **reference_document.docx** - Institutional formulary digest containing monographs for the medications referenced in the primary document, plus several additional monographs for medications not referenced (intentional, to provide retrieval haystack). This is the authoritative source against which the primary document should be verified.

LLM access: use any provider you prefer (OpenAI, Anthropic, a local model via Ollama, etc.)

6. Deliverables

- A public or private GitHub repository (invite the reviewer if private).
- A README at the repository root containing setup instructions, architectural overview, schema diagram or description, key design decisions and tradeoffs, bonus points attempted and known issues.
- Seed credentials (a username and password) in the README so the reviewer can log in without signing up.

Good luck. We are looking forward to seeing what you build.